CASE STUDY · SHOWCASE EDITION

Legacy SAS → CDP Migration

Volume 3

Assurance, Delivery & Lessons

How correctness was proven, how the work was run safely, and what transfers

WHAT THIS VOLUME COVERS

The two-stage QC framework, the test harness, runtime findings, the production safety runbook, governance and the decision register, the iteration timeline and roles, outcomes, twenty lessons in full, eight takeaways and a phase-by-phase playbook for future engagements.

ABOUT THIS DOCUMENT

"Meridian Media" is a hypothetical organisation. All client names, data values, file names and code identifiers have been anonymised or generalised to protect client identity and intellectual property. Figures are illustrative and rounded; the method and sequence of work are described as executed.

Client Analytics · Project Documentation Set (3 volumes)

Contents

1. Quality Assurance Framework

A clean execution log proves only that the code ran. Every run passed two stages of assurance before any result was reported.

1.1 Stage 1 — self-audit inside the CDP

Check	Rule	What it catches
Row-count integrity	Output rows must equal the requested audience exactly	Merges that append instead of join; duplicated keys
Population census	Every column classified populating / all-zero / all-blank	Silently empty variables; the zero list is the defect queue
Attribute comparison	Type, length and format compared to the legacy structure	Cosmetic versus material differences
Source reconciliation	Suspect variables re-derived directly against the CDP	Engine output that differs from what the source supports

1.2 Stage 2 — validation against legacy

1. **Normalise dates.** Convert both outputs from days-since-build to calendar dates using each output's own build date.
2. **Match on the overlap.** Compare only customers present in both populations.
3. **Band the results.** Group variables by match rate against the agreed 90% threshold.
4. **Separate defect from difference.** Extra months of activity and re-classified products are expected variance and are labelled as such, not chased as bugs.

> **THE GATE**
> Nothing was signed off on a clean log. A run with zero errors still carried 31 all-zero variables — the census, not the log, caught them.

2. Test Harness

The production audience was about 600,000 customers and a full run took about an hour. A purpose-built harness of about 1,000 customers made verification fast without compromising its value.

2.1 Design

- **Selection measure** — customers ranked by the number of variables actually carrying data. Non-missing is not enough, because the engine zero-fills numerics that match nothing.
- **Threshold, not count** — a score threshold rather than a fixed count, so customers tied at the boundary were not dropped arbitrarily.
- **Isolation** — registered under its own audience keycode and written to its own output name.
- **Fidelity check** — the harness reproduced the full run's error profile: the same ~5,000-error cascade and the same 13 missing variables.

2.2 Limits

A harness of the richest records proves code paths; it misstates fill rates. Three of the final six variables looked empty on the harness and populated on the full audience. Final population was therefore always measured on the full production audience.

3. Performance and Runtime

Observation	Value
Queries per full run	~2,400
Fixed overhead per query	~5 seconds regardless of rows returned
Runtime on ~600K audience	~1 hour
Runtime on ~1K harness	~1 hour
Share of runtime in 4–6 second queries	57%

The uniform per-query floor is scheduling and round-trip latency, not data volume. Runtime scales with the number of variables requested, not the number of customers.

3.1 Levers in order of payback

1. **Subset the variable list for debugging** — validate a fix on the 10–20 variables it touches; minutes instead of an hour.
2. **Run variable families in parallel** — independent categories in separate sessions with distinct staging names.
3. **Stage the heaviest views once** — pre-filter the largest views to the audience and query the small copy.

4. Production Safety Runbook

All work ran against a shared production platform. The following rules applied to every change and every run.

4.1 Before a run

- Confirm connection, schema and code path — test and production share naming patterns.
- Confirm audience keycode, source identifier and output name; test runs never write to production output names.
- Copy the last good output to a protected name.
- Confirm no stale staging tables exist (the orchestrator guard also enforces this).
- Confirm reference codes in the CDP are known to the lookups.

4.2 Deploying a change

- Apply the change to the real source file; preserve encoding and line endings.
- Diff against the original; only intended lines may differ.
- Check macro definitions balance.
- Run a fresh-session compile check: every expected bridge macro exists.
- Run on the harness; confirm by value; then promote.

4.3 After a run

- Locate the run's own (redirected) log and search from the first error.
- Run the row-count check and population census.
- If the output is missing, check whether the previous output was renamed to a backup before re-running.

5. Governance and Decision Register

Engineering decides whether code does what it says; the business decides what it should say. Any fix that changes the meaning of a number went to its owner with evidence, before/after impact and a recommendation.

ID	Decision	Evidence	Outcome
G01	Email-tenure date from two fields	Field distribution clustered on load dates	Agreed with platform team and modelling lead; implemented
G02	Blank channel code means non-web	Blank on >99.9% of paid non-web orders	Raised as assumption; confirmed by client; implemented
G03	Paid-only shipment counts	~7% of counted orders unpaid	Signed off by business; released
G04	Retired product identifiers	~10% of title family surviving in CDP	Client supplied current identifiers; lookup added
G05	Unseen reference code	Integrity-check stop	Parked in neutral group; moved after client confirmed family
G06	Port one unmapped legacy variable	Legacy structure comparison	Approved; ported as one recipe on an existing builder

6. Iteration Timeline and Roles

When	Iteration	Lead role	Outcome
Month 1	Parallel run in production	Analytics engineer	344 / 475; ~5,000 errors
Month 2	Bridge layer and reference-sample check	Engineer, QC lead	423 / 475
Month 3, week 1	Variable triage by root-cause family	Engagement lead	6 families
Month 3, week 2	NULL-safe exclusions	Analytics engineer	436 / 475; errors to 3
Month 3, week 3	Date re-sourcing and mapping correction	Analytics engineer	464 / 475; errors to 0
Month 3, week 4	Recent build date	QC lead	469 / 475
Month 4	Final-six closure	All roles	475 / 475

Ownership was split so that whoever fixed a defect was never the only one validating it: the analytics engineer remediated, the QC lead certified by census and reconciliation, and the engagement lead governed meaning changes with the client.

7. Outcomes

Outcome	Result
Execution errors	~5,000 → 0
Variables populating	344 → 475 of 475 (100%)
Platform-specific code	Isolated in 2 bridge files
Recipe logic rewritten for the platform	None
Sampled magazine revenue	~35% understatement removed
Variables waived	None
Production disruption	None

7.1 How the final six were closed

Variable(s)	Judgement	Resolution
1 channel variable	Code defect plus business rule	NULL-safe not-equal swept into sign slots; client confirmed blank = non-web; paid filter signed off
2 promotion variables	Reference-data drift	Client-supplied current identifiers mapped in the bridge
3 activity-sparse variables	Validation scope and one missing recipe	Two validated on the full audience; one ported from legacy

8. Twenty Lessons

Each lesson is tagged by who surfaced it: the team, the AI, or both working together.

#	Lesson	Surfaced by
01	Give it evidence, not adjectives	AI + Team
02	Push back on the first answer	Team
03	Diagnose at the source, not in the output	AI
04	Fix the generator, not the symptom	AI
05	A clean log is not a correct output	AI + Team
06	Separate 'missing' from 'wrong'	AI + Team
07	Invest early in a feedback loop	Team
08	Know when it is data — then close it	AI + Team
09	Read the log the run actually wrote	AI
10	Guard shared state before every run	AI + Team
11	Edit the real file, then prove it compiled	Team
12	Keep the client's code recognisably theirs	Team
13	Check key domains before merging	AI
14	Normalise before you compare	AI + Team
15	Measure where the time goes	AI
16	The business owns what a number means	Team
17	Expect reference data to drift	AI + Team
18	Test the fix, not just the variable	AI + Team
19	Sweep the whole defect class	AI
20	Write it down as you go	Team

Lesson 01 — Give it evidence, not adjectives

What happened. A complaint that magazine revenue was 'too low' produced a list of plausible causes. The emitted predicate, the type-code distribution and the revenue summed both ways produced the diagnosis in one pass.

The practice. Brief people and models with artefacts — counts, log lines, column profiles — not impressions. The same evidence then goes straight into the client update.

Lesson 02 — Push back on the first answer

What happened. Twice the first explanation was coherent, confident and wrong: an 'ambiguous join' that was really an empty column, and a key switch that would have broken 25 working variables.

The practice. Ask what evidence would disprove an explanation, then go and look. One read-only query is cheaper than a wrong rebuild.

Lesson 03 — Diagnose at the source, not in the output

What happened. Blank output columns traced to an empty source column, a key-domain mismatch and blank filter values — none visible in the code.

The practice. For every suspect column: is it populated, do the keys share a domain, do the filter values exist?

Lesson 04 — Fix the generator, not the symptom

What happened. One exclusion branch in three builders served 118 variables. Six lines fixed all of them.

The practice. In generated-SQL engines, find the shared builder and fix it there.

Lesson 05 — A clean log is not a correct output

What happened. A zero-error run still held 31 all-zero variables and understated revenue.

The practice. Use a population census and value reconciliation as the acceptance gate; treat the log as proof of completion only.

Lesson 06 — Separate 'missing' from 'wrong'

What happened. Absent, zero-filled and understated variables were all reported identically and had unrelated causes.

The practice. Classify the symptom before choosing the test.

Lesson 07 — Invest early in a feedback loop

What happened. A full run took an hour; a ~1K harness reproduced the full failure profile.

The practice. Build the harness in week one, isolate it, and never measure fill rates on it.

Lesson 08 — Know when it is data — then close it

What happened. Blank channel codes and retired product identifiers could not be fixed in code alone. Both were closed by evidenced proposals the client confirmed.

The practice. Rule out build date and sample scope; then escalate with evidence and a proposal, not a complaint.

Lesson 09 — Read the log the run actually wrote

What happened. The visible log reported a missing dataset; the redirected log held the real cause — an integrity-check failure.

The practice. Find the run's own log and start from the first error, not the last.

Lesson 10 — Guard shared state before every run

What happened. A table left by an aborted run caused the next run to finish 'successfully' with zero rows.

The practice. Assume the last run died halfway; make every run safe to start from that state.

Lesson 11 — Edit the real file, then prove it compiled

What happened. A one-macro patch saved over the whole bridge file erased every mapping.

The practice. Edit the source byte-for-byte, diff it, and compile-check in a fresh session after deployment.

Lesson 12 — Keep the client's code recognisably theirs

What happened. Two bridge files absorbed every platform difference; recipes stayed intact.

The practice. Translate at the boundary; change recipes only for business-requested meaning changes.

Lesson 13 — Check key domains before merging

What happened. Two look-alike keys turned a join into an append; output grew by half.

The practice. Test key overlap; enforce output rows = audience size on every run.

Lesson 14 — Normalise before you compare

What happened. Outputs built a year apart look wrong in raw form because dates are offsets from different build dates.

The practice. Convert to calendar dates, compare on the overlapping population, band expected drift.

Lesson 15 — Measure where the time goes

What happened. A 570-fold smaller audience barely changed runtime; query count set the clock.

The practice. Time queries before optimising; subset variables for debugging.

Lesson 16 — The business owns what a number means

What happened. Paid-only counts, blank-means-non-web and two-field dates all changed meaning.

The practice. Prove the effect; let the owner decide; record the decision.

Lesson 17 — Expect reference data to drift

What happened. New codes stopped runs; retired codes silently emptied variables.

The practice. Keep lists in one lookup, check against the source each run, fail loudly on unknowns.

Lesson 18 — Test the fix, not just the variable

What happened. Two fixes introduced defects: a type-unsafe NULL guard and a macro-parsed operator word.

The practice. A generator fix is a release: test across types and operators before promotion.

Lesson 19 — Sweep the whole defect class

What happened. The first NULL fix covered exclusion slots but not sign slots; the last channel variable stayed at zero.

The practice. Name the defect class, enumerate every site, fix them together and publish the impact list.

Lesson 20 — Write it down as you go

What happened. Fix and decision registers fed every status update and this case study.

The practice. Record each fix and decision when it happens, not afterwards.

9. Eight Takeaways for Future Engagements

#	Takeaway	What it changes
01	Fund discovery as a phase	Undocumented estates get a paid comprehension phase before estimates
02	Absorb differences in a bridge layer	Default migration pattern; client code stays theirs
03	Audit NULL and type semantics on day one	Reusable pre-flight check of every not-equal filter and date conversion
04	Make the census the acceptance gate	'Done' means population and reconciliation, not error counts
05	Build the harness and guards in week one	Small audience, isolated outputs, stale-table guard, compile checks
06	Close gaps — do not waive them	Rule out build date and sample scope, then escalate with a proposal
07	Route meaning changes to the business	Evidence, impact, recommendation, recorded decision
08	Size runtime by queries, not rows	Plan parallelism and staging around query count and latency

10. Reusable Playbook

10.1 Phase 0 — mobilise

- Agree that discovery is a funded phase.
- Agree the definition of done: population census, reconciliation threshold, sign-off owner.
- Identify the business owner for variable meaning and the platform owner for data availability.

10.2 Phase 1 — discover

- Parse the include chain and record load order.
- Map the macro call graph; capture emitted SQL from a baseline run.
- Build variable lineage: table, slots, keys, window, aggregation.

10.3 Phase 2 — pre-flight on the target platform

- Profile every filter column for NULLs; list every not-equal site in every slot.
- Profile date column types.
- Compare reference-code lists against the target.
- Test key overlap for every join key.

10.4 Phase 3 — build the safety net

- Create the harness audience and isolated outputs.
- Add stale-state guards and the fresh-session compile check.
- Set up the census and normalised reconciliation.

10.5 Phase 4 — remediate

- Walk the derivation path for each defect; fix at the generator; one change at a time.
- Record every fix and decision as it happens.
- Escalate data gaps with evidence and a proposal.

10.6 Phase 5 — prove and hand over

- Census and reconcile on the full production audience.
- Obtain sign-off for every meaning change.
- Hand over architecture, lineage, bridge specifications and both registers.

> **CLOSING**
> Discovery was the deliverable. Understanding the system well enough to reason about it — not any single fix — is what took the pipeline from ~5,000 errors to 475 of 475, and it is what transfers to the next migration.